

Sommaire

- [1. Introduction](#)
- [2. Vue d'ensemble de l'architecture](#)
- [3. Cycle d'exécution \(runtime\)](#)
- [4. Configuration des modules et locales](#)
- [5. Store Redux et middlewares](#)
- [6. Thème et styles](#)
- [7. Localisation \(i18n\)](#)
- [8. Démarrage, build et déploiement](#)
- [9. Modules: comment ajouter/éditer](#)
- [10. Dépendances clés](#)
- [11. Résolution de problèmes courants](#)
- [12. Arborescence minimale utile](#)
- [13. Références rapides \(extraits\)](#)
- [14. Bonnes pratiques](#)
- [15. Commandes utiles](#)
- [16. Flux d'authentification \(frontend\)](#)
- [17. Exemple concret de contribution de module](#)

1. Introduction

Ce dépôt assemble l'interface web openIMIS à partir de modules frontend publiés sous le scope `@openimis`. Il utilise React 17, Material-UI v4 et Redux, avec un système de modules dynamiques configurés via `openimis.json` et consommés au runtime.

Objectifs de ce document:

- Expliquer l'architecture, le cycle d'exécution et la configuration.
- Indiquer comment lancer, développer, thémer, localiser et livrer.

2. Vue d'ensemble de l'architecture

- Entrée applicative: `src/index.js` démarre l'app, charge la configuration GraphQL des modules, instancie le `ModulesManager` et monte Redux.
- Système de modules: `src/modules.js` déclare une liste de packages `@openimis/*` et un chargeur `loadModules(cfg)` qui importe chaque module et collecte ses contributions (reducers, middlewares, menus, écrans, etc.).
- Gestionnaire de modules: `src/ModulesManager.js` encapsule l'accès aux contributions, références, rapports, contrôles d'affichage (masquage de champs), et versions.
- Store Redux: `src/helpers/store.js` assemble les reducers fournis par les modules + middlewares (thunk, redux-api-middleware...).
- Thème UI: `src/helpers/theme.js` définit la palette, la typographie et des tokens de styles applicatifs.
- Localisation: `src/LocalesManager.js` et `src/locales.js` gèrent langues et fichiers de traduction. Les messages de base sont dans `src/translations/ref.json`.
- Configuration d'assemblage: `openimis.json` déclare les modules et les locales à intégrer (source de vérité pour `yarn load-config`).
- Serveur statique de prod simple: `server.js` (Express) sert le build.

3. Cycle d'exécution (runtime)

1. L'app démarre dans `src/index.js` et affiche un loader.
2. Elle interroge l'API GraphQL `baseApiUrl/graphql` (provenant de `@openimis/fe-core`) pour récupérer `moduleConfigurations` (config JSON et contrôles de champs par module).
3. Ces configurations sont transmises au `ModulesManager`, qui:
 - charge les modules via `loadModules(cfg)`,
 - agrège contributions: reducers, middlewares, menus, références, rapports,
 - prépare caches `controls`, `refs`, `reports`.
4. On construit le store Redux à partir des reducers et middlewares collectés.
5. On rend l'App de `@openimis/fe-core` avec le `ModulesManagerProvider`, le thème MUI, la gestion des dates et des messages.

4. Configuration des modules et locales

- `openimis.json` déclare:
 - `locales` : langues disponibles (par ex. `en-GB`, `fr-FR`) avec fichiers associés,
 - `modules` : liste et provenance de chaque module (URL git + branche/tag, ou version npm).
- `package.json` référence également les modules dans `dependencies` pour la résolution au build.
- Commande utile: `yarn load-config` régénère imports/locales à partir de `openimis.json`.

Extrait simplifié d' `openimis.json` (modules):

```
{
  "modules": [
    { "name": "CoreModule", "npm": "@openimis/fe-core@https://github.com/...#release/25.04" },
    { "name": "InsureeModule", "npm": "@openimis/fe-insuree@https://github.com/...#release/25.04" }
  ]
}
```

5. Store Redux et middlewares

- `src/helpers/store.js` construit le store avec:
 - reducers: fournis par `ModulesManager.getContribs("reducers")`,
 - middlewares: `redux-thunk`, `redux-api-middleware` + ceux des modules.
- L'état persistant (si utilisé) provient de `helpers/localStorage.js`.

6. Thème et styles

- `src/helpers/theme.js` définit palette, typo, couleurs de tableaux, formulaires, dialogues, etc.
- Les composants Material-UI v4 utilisent ce thème via `MuiThemeProvider`.

7. Localisation (i18n)

- `src/LocalesManager.js` et `src/locales.js` indiquent les locales disponibles et le mapping vers les fichiers de traduction.
- Les modules peuvent ajouter leurs propres messages via leurs contributions.
- `openimis.json` pilote la liste des locales intégrées au bundle.

8. Démarrage, build et déploiement

Pré-requis: Node 16.x, Yarn.

- Développement:
 - `yarn load-config` (génère imports/modules selon `openimis.json`)
 - `yarn install`
 - `yarn start` (CRA avec proxy vers `http://localhost:8000`)
- Build production:
 - `yarn build` (génère `build/`)
 - Servir via `server.js` (Express) ou tout serveur statique
- Docker: Voir `docs/WINDOWS_DOCKER.MD` et `docs/LINUX_DOCKER.MD` .

9. Modules: comment ajouter/éditer

- Ajouter un module existant: le référencer dans `openimis.json` + `package.json` , puis `yarn load-config` et `yarn install` .
- Développer un module localement:
 - cloner le repo du module à côté de l'assembly,
 - dans le module: `yarn install && yarn build && yarn link` ,
 - dans l'assembly: `yarn remove @openimis/fe-xxx` puis `yarn link "@openimis/fe-xxx"` .

Les contributions d'un module (exemples fréquents):

- reducers, middlewares, écrans, routes, entrées de menu, références (GraphQL projections), rapports.

10. Dépendances clés

- React 17, Material-UI v4, Redux, Redux Thunk, Redux API Middleware
- Moment et `@date-io/moment` pour les dates
- `@openimis/fe-core` : fournit `App` , constantes, entêtes API, base de l'architecture.

11. Résolution de problèmes courants

- Page vide ou loader infini:
 - vérifier versions backend/frontend compatibles, base de données à jour.
- Erreur 400 au login/home en prod:
 - variables d'env côté backend (`SITE_ROOT`), valeur `proxy` dans `package.json` .

12. Arborescence minimale utile

- `src/index.js` : bootstrap de l'app
- `src/modules.js` : liste/chargeur des modules
- `src/ModulesManager.js` : agrégation des contributions et accès config
- `src/helpers/store.js` : store Redux
- `src/helpers/theme.js` : thème Material-UI
- `openimis.json` : modules + locales

- `server.js` : serveur Express (prod simple)

13. Références rapides (extraits)

Entrée applicative (`src/index.js`):

```
ReactDOM.render(<AppContainer />, document.getElementById("root"));
```

Chargement des modules (`src/modules.js`):

```
export function loadModules(cfg = {}) {  
  const loadedModules = [];  
  loadedModules.push(require("@openimis/fe-core").CoreModule(cfg["fe-core"] || {}));  
  // ... autres modules  
  return loadedModules;  
}
```

Accès aux contributions (`src/ModulesManager.js`):

```
getContribs = memoize((key) => this.modules.reduce((acc, m) => [...acc,  
  ...ensureArray(m[key])], []));
```

14. Bonnes pratiques

- Centraliser les constantes et types dans les modules.
- Préférer des noms explicites (reducers/actions séparés par domaine).
- Éviter les états globaux non normalisés; utiliser des sélecteurs mémorisés.
- Respecter les interfaces fournies par `@openimis/fe-core` (menus, structures de contributions).

15. Commandes utiles

- `yarn load-config` — régénère imports/locales à partir d' `openimis.json` .
- `yarn start` — dev server CRA (proxy backend port 8000).
- `yarn build` — build de production.

Pour plus de détails, voir `README.md` et la documentation des modules individuels.

16. Flux d'authentification (frontend)

Principe général (basé sur `@openimis/fe-core`):

- Le formulaire de login envoie les identifiants vers le backend (généralement via un endpoint exposé par l'API du backend openIMIS).
- En cas de succès, un jeton/session est stocké côté frontend (souvent via des utilitaires de `@openimis/fe-core`) et les en-têtes d'API (`apiHeaders()`) incluent l'authentification pour les requêtes suivantes.
- Le menu, les routes et les fonctionnalités s'adaptent au profil/aux droits de l'utilisateur.

Points d'intégration côté frontend:

- Les écrans/menus de login/logout/profil proviennent du module `@openimis/fe-core` et des modules de profil.
- Les appels API passent par `baseApiUrl` + `apiHeaders()` (cf. `src/index.js`), assurant la propagation du contexte d'authentification.

Bonnes pratiques:

- Ne jamais stocker d'identifiants en clair.
- Centraliser l'accès au token/session et gérer proprement l'expiration.
- Protéger les routes privées côté UI et masquer les entrées de menu sans droit.

17. Exemple concret de contribution de module

Objectif: ajouter une entrée de menu et un reducer via un module custom `@openimis/fe-mymodule`.

Étapes:

1. Dans le module, exposer une fonction `MyModule(cfg)` qui retourne un objet de contributions, par exemple:

```
export const MyModule = (cfg) => ({
  // Reducers
  reducers: [
    { key: 'myDomain', reducer: myDomainReducer }
  ],
  // Entrées de menu (exemple)
  'mymodule.MainMenu': [
    { text: 'My Feature', icon: MyIcon, route: '/mymodule' }
  ],
});
```

2. Publier/relire le module:

- En local: `yarn link` dans le module, puis `yarn link "@openimis/fe-mymodule"` dans l'assembly.
- Ou via `openimis.json` + `package.json` avec une version/tag.

3. Dans `openimis.json`, ajouter le module puis exécuter `yarn load-config` et `yarn install`.

4. Lancer `yarn start` et vérifier:

- Le reducer `myDomain` est présent dans le store.
- L'entrée de menu "My Feature" apparaît et ouvre la route déclarée.